COS101 Practical — Week 1

Introduction to Python and Problem Solving

Programme: B.Sc. Software Engineering
Level: 100
Suggested laboratory duration: 3 hours
Prerequisite: Basic ability to use a computer; no previous programming experience required.

1. Learning objectives

By the end of this practical, students should be able to:

1. Explain what a program is and describe the purpose of Python.
2. Identify the Python interpreter, interactive shell, and script editor.
3. Create, save, and execute a Python file.
4. Display text and simple calculation results using `print()`.
5. Write useful comments.
6. Recognise basic syntax errors and indentation problems.
7. Describe a solution using an algorithm and pseudocode.
8. Identify the input, processing, and output of a simple problem.
9. Produce a basic personal introduction program independently.

---

2. Materials required

Each student or pair of students should have:

- A computer with Python 3 installed.
- IDLE, which is commonly included with Python.
- A dedicated folder for COS101 practical work.
- A notebook for algorithms, observations, and exercise answers.

---

3. Introduction: What is programming?

Programming is the process of writing instructions that a computer can execute to perform a task.

Examples of tasks that programs can perform include:

- Calculating a student's total score.
- Generating a receipt.
- Recording attendance.
- Converting temperatures.
- Calculating the area of a room.
- Displaying messages to a user.

A computer follows the instructions provided. If the instructions contain mistakes, the program may fail or produce an incorrect result.

Consider a program intended to calculate the area of a rectangle. It must:

1. Obtain the length.
2. Obtain the width.
3. Multiply the length by the width.
4. Display the answer.

Writing the Python code is only one part of the work. We must first understand the problem and decide how to solve it.

3.1 What is Python?

Python is a programming language used to express instructions in a form that people can read and a Python interpreter can execute.

It is used for many kinds of work, including:

- General application development.
- Automation.
- Web development.
- Mathematical calculations.
- Data analysis.
- Scientific computing.

In COS101, we will use Python to develop programming foundations and solve simple practical problems.

3.2 The Python interpreter

The **Python interpreter** is the software that executes Python instructions.

The editor helps you write code. The interpreter runs the code.

Term	Meaning
Source code	The instructions written by the programmer.
Editor	A tool used to create and modify source code.
Interpreter	The software that executes Python code.
Output	The result produced by a program.
Error message	Information explaining a problem encountered while running code.

---

4. Preparing the working environment

4.1 Confirm that Python is available

On a Windows computer:

1. Open the Start menu.
2. Search for **IDLE**.
3. Open the application associated with Python 3.

If IDLE opens successfully, you should see a window containing a prompt similar to:

>>>

The exact version information above the prompt may differ between computers.

On a computer where Python is accessed through a terminal, use:

```
python --version
```

Some computers use:

```
python3 --version
```

On Windows, the Python launcher may also be available:

```
py --version
```

Use whichever command successfully identifies your PC's Python 3 installation.

4.2 Create your practical folder

Create the following folder structure inside your Documents or Desktop folder or another location of your chosen:

COS101_Practicals/
  Week01/

Save all work for this session inside `Week01`.

Use clear filenames, for example:

- `hello.py`
- `personal_introduction.py`
- `rectangle_area.py`

The `.py` extension identifies a Python source file.

Avoid filenames such as `python.py`, which can conflict with tools or cause confusion, and names containing unnecessary punctuation.

---

5. Working in the interactive shell

The interactive shell allows you to enter a Python instruction and immediately see its result.

This is useful for experimenting with short instructions.

Practical Activity 1: Display a message

At the `>>>` prompt, type:

print("Hello, world!")

Press **Enter**.

Expected output:

Hello, world!

Do not type `>>>`. It is the shell prompt, not part of your code.

5.1 Understanding the instruction

print("Hello, world!")

Part	Purpose
`print`	A built-in function that displays information.
`(` and `)`	Enclose the information passed to the function.
`"Hello, world!"`	A string: a piece of text enclosed in quotation marks.

Python is case-sensitive. The built-in function is named `print`, using lowercase letters.

Practical Activity 2: Display additional messages

Enter these instructions one at a time:

```
print("Welcome to COS101 Practical.")
print("I am learning Python.")
print("Programming begins with clear instructions.")
```

Observe that each call to `print()` displays its output on a new line.

5.2 Single and double quotation marks

Both examples below are valid:

```
print("Python is interesting.")
print('Python is interesting.')
```

The opening and closing quotation marks must match.

This is invalid:

```
print("Python is interesting.')
```

When text contains an apostrophe, using double quotation marks is convenient:

```
print("I am a student, and I'm learning Python.")
```

Practical Activity 3: Try simple calculations

Enter:

2 + 3

Expected result:

5

Then enter:

```
10 - 4
6 * 7
20 / 5
```

Expected results, in order:

6
42

4.0

The symbols used here are arithmetic operators.

Operator	Meaning	Example	Result
`+`	Addition	`2 + 3`	`5`
`-`	Subtraction	`10 - 4`	`6`
`*`	Multiplication	`6 * 7`	`42`
`/`	Division	`20 / 5`	`4.0`

For now, observe that ordinary division produces a floating-point result, such as `4.0`. Data types and arithmetic will be covered more fully next week.

---

6. Creating and running a Python script

The shell is useful for short experiments. A **script** allows you to save several instructions in a file and run them together.

6.1 Shell mode and script mode

Feature	Interactive shell	Script
Where code is entered	At the `>>>` prompt	In an editor
Execution	Usually one instruction at a time	Runs the saved program

Saving	Does not automatically create a reusable program file	Code is saved in a `.py` file
Typical use	Quick experiments	Practical assignments and applications

Practical Activity 4: Create your first script in IDLE

1. In IDLE, select **File** → **New File**.
2. A blank editor window will open.
3. Type:

```
print("Hello, world!")
print("Welcome to COS101 Practical.")
print("This is my first saved Python program.")
```

4. Select **File** → **Save As**.
5. Open your `Week01` folder.
6. Save the file as:

hello.py

7. Select **Run** → **Run Module**, or press **F5**.
8. If prompted to save, save the file.
9. Observe the output in the shell window.

Expected output:

```
Hello, world!
Welcome to COS101 Practical.
This is my first saved Python program.
```

6.2 Running a script from a terminal

If using a terminal, first open the directory containing the saved file. Then use the Python command supported by your computer:

```
python hello.py
```

or:

```
python3 hello.py
```

or, on Windows where the launcher is available:

```
py hello.py
```

6.3 Why saving matters

If you change the code in the editor but run an older saved version, the output may not reflect your changes.

Develop this habit:

1. Edit.
2. Save.
3. Run.
4. Inspect the output.

6.4 A useful difference between the shell and scripts

In the interactive shell, entering:

2 + 3

displays:

5

However, a script containing only:

2 + 3

does not display that result.

To display it from a script, use:

```
print(2 + 3)
```

---

7. Using `print()` effectively

7.1 Printing text

```
print("Department of Software Engineering")
```

Expected output:

```
Department of Software Engineering
```

7.2 Printing a number

```
print(100)
```

Expected output:

100

Numbers do not need quotation marks when used as numbers.

7.3 Printing the result of a calculation

```
print(8 + 4)
```

Expected output:

12

Compare this with:

```
print("8 + 4")
```

Expected output:

8 + 4

The first instruction calculates an answer. The second displays text.

7.4 Printing a label with a result

Separate the items with a comma:

```
print("Total:", 8 + 4)
```

Expected output:

Total: 12

By default, `print()` places a space between the items.

7.5 Printing a blank line

```
print("COS101 Practical")
print()
print("Week 1: Introduction to Python")
```

Expected output:

COS101 Practical

Week 1: Introduction to Python

Practical Activity 5: Create a simple course display

Save the following as `course_display.py`:

```
print("================================")
print("COS101 PRACTICAL")
print("Introduction to Python")
print("================================")
print()
print("Programme: Software Engineering")
print("Level: 100")
print("Week: 1")
print()
print("Today, I will create and run a Python program.")
```

Run it, then change the final message to a sentence of your own.

---

8. Writing comments

A comment is a note written inside source code.

In Python, # begins a comment outside a string. The interpreter ignores the rest of that line.

Example:

Display a welcome message
`print("Welcome to COS101.")`

Comments can also appear after an instruction:

`print("Week 1")` # Display the practical week

8.1 Why comments are useful

Comments can:

- Explain the purpose of a program.
- Explain a decision that is not obvious from the code.
- Help another person understand the work.

Avoid comments that merely repeat a clear instruction without adding useful information.

A useful introductory comment might be:

This program displays a student's basic academic information.

Practical Activity 6: Add comments to your script

Open `course_display.py` and add:

```
# Course: COS101 Practical
# Week: 1
# Purpose: Display basic information about the practical
session.
```

Save and run the program.

Observation: The comments should not appear in the output.

Also compare:

```
# Welcome to Python
```

with:

```
print("# Welcome to Python")
```

In the second example, # is inside quotation marks, so it is part of the displayed text.

---

9. Understanding syntax and indentation

9.1 What is syntax?

Syntax is the set of rules used to write valid instructions in a programming language.

For example, this is valid:

```
print("Hello")
```

This is invalid:

```
print("Hello"
```

The closing parenthesis is missing.

9.2 Indentation

Indentation means placing spaces at the beginning of a line.

Python uses indentation to identify groups of instructions called **blocks**. You will use these blocks when studying decisions, loops, and functions.

For this week's simple scripts, begin each instruction at the left margin.

Correct:

```
print("First message")
print("Second message")
```

Incorrect at the top level of a script:

```
print("First message")
     print("Second message")
```

The second line has an unexpected indentation.

When blocks are introduced later, the class should use four spaces per indentation level.

---

10. Common errors and how to correct them

Errors are a normal part of learning programming. The important skill is to investigate them carefully.

Run each faulty example separately, read the message, and correct it.

Faulty code	Problem	Corrected code
`Print("Hello")`	The function name uses an uppercase `P`.	`print("Hello")`
`print("Hello)`	Missing closing quotation mark.	`print("Hello")`
`print("Hello"`	Missing closing parenthesis.	`print("Hello")`
`print(Hello)`	Python treats `Hello` as a name rather than text; it has not been defined.	`print("Hello")`
`print("Hello")`	Unexpected indentation at the top level of a script.	`print("Hello")`

The exact wording of an error message can differ between Python versions.

10.1 Syntax errors

A syntax error occurs when Python cannot understand the structure of the code.

Example:

```
print("Welcome"
```

10.2 Runtime errors

A runtime error occurs when a syntactically valid instruction fails during execution.

Example:

```
print(10 / 0)
```

This raises a `ZeroDivisionError`.

Another example is:

```
print(Hello)
```

If `Hello` has not been defined, this raises a `NameError`.

10.3 Logical errors

A logical error occurs when the program runs but the instructions produce the wrong answer.

Suppose the area of a rectangle should be calculated using multiplication. This program uses addition:

```
print("Area:", 8 + 5)
```

It displays:

Area: 13

The program runs, but the formula is wrong.

The correct instruction is:

```
print("Area:", 8 * 5)
```

Expected output:

Area: 40

10.4 A simple debugging procedure

When a program fails:

1. Read the error message.
2. Identify the file and line mentioned.
3. Inspect that line and the line immediately before it.
4. Check spelling, quotation marks, parentheses, and indentation.
5. Make one relevant correction.
6. Save and run again.
7. Confirm that the output matches the intended result.

Debugging is the process of finding and correcting errors.

---

11. Problem solving before coding

Before writing code, answer these questions:

1. What problem am I solving?
2. What information is needed?
3. What calculation or action must be performed?
4. What result should be produced?
5. How can I check that the result is correct?

This helps prevent writing a program that runs successfully but solves the wrong problem.

11.1 The input–process–output model

Many introductory programs can be understood using three parts:

Part	Question	Rectangle example
Input	What information is needed?	Length and width

Process	What must be done with the information?	Multiply length by width
Output	What result should be produced?	Area

Input does not always mean information typed by a user. For this week's exercises, values can be supplied directly in the code. Keyboard input will be introduced next Week.

11.2 Example: Calculating a purchase total

Problem: A student buys three notebooks at ₦500 each. Calculate the total cost.

Part	Description
Input	Quantity: 3; price per notebook: ₦500
Process	Multiply quantity by unit price
Output	Total cost: ₦1,500

Python implementation:

```
print("Total cost in naira:", 3 * 500)
```

Expected output:

Total cost in naira: 1500

---

12. Algorithms

An **algorithm** is a finite sequence of clear steps for solving a problem.

A useful algorithm should:

- State the steps clearly.
- Put the steps in the correct order.
- Use the required information.
- Produce the intended result.

- Terminate after the task is complete.

12.1 Example: Rectangle area algorithm

Problem: Calculate the area of a rectangle with length 8 metres and width 5 metres.

Algorithm:

1. Start.
2. Use 8 metres as the length.
3. Use 5 metres as the width.
4. Multiply the length by the width.
5. Display the area in square metres.
6. Stop.

Manual calculation:

$$A = L \times W = 8 \times 5 = 40m^2$$

12.2 Why order matters

An instruction to display an answer should come after the answer has been calculated.

Likewise, if a calculation requires length and width, those values must be available before the calculation is performed.

---

13. Pseudocode

Pseudocode describes the steps of an algorithm using structured, readable language.

It is used for planning. It is not Python code and should not be entered into the Python shell.

There is no single compulsory pseudocode syntax. In this course, we will use simple keywords such as:

- START
- SET
- CALCULATE
- DISPLAY
- STOP

13.1 Rectangle area pseudocode

```
START
    SET length TO 8
    SET width TO 5
    CALCULATE area AS length × width
    DISPLAY area
STOP
```

13.2 Python implementation for this Week

We have not yet formally introduced variables, so use the fixed values directly:

```
# Calculate the area of a rectangle with fixed dimensions.
print("Rectangle Area Calculation")
print("Length: 8 metres")
print("Width: 5 metres")
print("Area:", 8 * 5, "square metres")
```

Expected output:

Rectangle Area Calculation
Length: 8 metres
Width: 5 metres
Area: 40 square metres

Next Week, we will store the dimensions in variables and allow the user to supply them.

13.3 Checking the answer

A program's output should be checked independently.

For this example:

- Manual result: $8 \times 5 = 40$.
- Program result: `40`.
- Unit: square metres.

The result and its unit are both correct.

---

14. Guided practical exercises

Complete the following exercises in order. Save each program in your `Week01` folder.

Exercise 1: Personal introduction

Filename: `personal_introduction.py`

Write a program that displays:

- Your full name.
- Your matriculation number, or “Not yet assigned” if unavailable.
- Your programme.
- Your level.
- One reason you want to learn programming.

Example structure:

```
# Purpose: Display my academic introduction.

print("PERSONAL INTRODUCTION")
print("====================")
print("Name: Your Full Name")
print("Matriculation Number: Not yet assigned")
print("Programme: Software Engineering")
print("Level: 100")
print()
print("I want to learn programming to solve everyday
problems.")
```

Replace the example details with your own information.

Check: Does the displayed information match what you intended?

Exercise 2: Text and calculation

Filename: `text_and_calculation.py`

Write:

```
print("7 + 3")
print(7 + 3)
print("Answer:", 7 + 3)
```

Before running the program, write down the output you expect.

Then run it and answer:

1. Why does the first line display `7 + 3`?
2. Why does the second line display `10`?
3. What does the comma do in the third line?

Exercise 3: Rectangle area

Filename: `rectangle_area.py`

A room has a length of 12 metres and a width of 7 metres.

Tasks:

1. Identify the input, process, and output.
2. Write a numbered algorithm.
3. Write pseudocode.
4. Write a Python program using `print()` and fixed values.
5. Check the result manually.

Expected area:

$12 \times 7 = 84m^2$

Exercise 4: Purchase total

Filename: `purchase_total.py`

A student buys four pens at ₦250 each.

Your program should display:

PURCHASE SUMMARY
Quantity: 4
Unit price: 250 naira
Total cost: 1000 naira

Use an arithmetic expression to calculate the total. Do not simply type `1000` as the answer.

Exercise 5: Debugging

The following program contains three mistakes:

```
Print("COS101 Practical")
print("I am learning Python)
    print("This is Week 1")
```

Tasks:

1. Identify each mistake.
2. Correct the program.
3. Save it as `debugging_exercise.py`.
4. Run it successfully.
5. Record the corrections in your notebook.

---

15. Independent assessment task

Title: My First Python Practical Portfolio

Submit these three working files:

1. `personal_introduction.py`
2. `rectangle_area.py`
3. `purchase_total.py`

Also submit a short practical record containing:

- The input–process–output analysis for the rectangle problem.
- Its algorithm and pseudocode.
- Expected and actual output for each program.
- One error you encountered and how you corrected it.

If you did not encounter an error during normal work, deliberately test one of the faulty examples in Section 10 and explain the correction.

Suggested marking guide

Criterion	Marks
Files saved with correct names and `.py` extensions	2
Personal introduction displays the required information	4

Rectangle program uses the correct calculation and unit	4
Purchase program calculates the total correctly	3
Input–process–output analysis, algorithm, and pseudocode	4
Useful comments and readable presentation	1
Explanation of an error and its correction	2
Total	**20**

---

16. Review questions

1. What is a computer program?
2. What does the Python interpreter do?
3. How does the interactive shell differ from a script editor?
4. What file extension is used for Python source files?
5. What is the purpose of `print()`?
6. What is the difference between `print("5 + 2")` and `print(5 + 2)`?
7. What symbol begins a Python comment?
8. Why is `Print("Hello")` incorrect?
9. What is an algorithm?
10. What is pseudocode?
11. Identify the input, process, and output for calculating the cost of five books at ₦2,000 each.
12. Can a program run successfully and still produce the wrong answer? Explain.

Instructor answer guide

Question	Expected answer

1	A set of instructions a computer can execute to perform a task.
2	It executes Python code.
3	The shell executes short instructions interactively; the editor creates saved programs.
4	`.py`
5	To display information.
6	The first displays text; the second calculates and displays `7`.
7	`#`
8	Python is case-sensitive; the function is named `print`.
9	A finite sequence of clear steps for solving a problem.
10	A readable, structured description of an algorithm used for planning.
11	Input: 5 books and ₦2,000 per book; process: multiplication; output: ₦10,000.
12	Yes. A logical error, such as using the wrong formula, can produce an incorrect result.

---

17. Take-home assignment

A classroom contains **six rows of chairs**, with **eight chairs in each row**.

1. State the problem in your own words.
2. Identify its input, process, and output.
3. Write a numbered algorithm.
4. Write pseudocode.
5. Create `classroom_chairs.py`.
6. Display the number of rows, chairs per row, and calculated total.
7. Include a comment explaining the program's purpose.
8. Compare the program's answer with your manual calculation.

Preparation for Week 2: Bring your saved files. You will extend these programs by using variables and accepting information from the user.